

Backtracking Study Guide

Backtracking Study Guide

1. Overview

Backtracking is a **depth-first search technique** used to explore all possible configurations of a problem. It builds solutions **incrementally**, abandoning ("backtracking") a path as soon as it determines that the path cannot lead to a valid solution.

Backtracking is ideal for:

- Combinatorial search
- Generating permutations/combinations
- Solving constraint problems (Sudoku, N-Queens)
- Exploring decision trees

It is often described as **DFS + undoing choices**.

2. How to Identify a Backtracking Problem

A problem is likely solved with backtracking if:

- You must **generate all valid solutions**.
- You must **search** through combinations, subsets, or permutations.
- You must **respect constraints** (e.g., cannot reuse, must satisfy rules).
- The solution involves **making a choice**, exploring, then **undoing** the choice.

Common signals:

- "Find all..."
 - "Generate all..."
 - "Return all valid..."
 - "Count the number of..."
-

3. Backtracking Template

This is the universal backtracking pattern:

```

result = []

def backtrack(state, choices):
    if state is a complete solution:
        result.append(copy of state)
        return

    for choice in choices:
        if choice is valid:
            make choice
            backtrack(updated state, updated choices)
            undo choice

backtrack(initial_state, initial_choices)
return result

```

Key components:

- **State:** partial solution
 - **Choices:** what you can try next
 - **Constraints:** rules that invalidate choices
 - **Goal:** when a solution is complete
-

4. Core Backtracking Patterns

4.1 [Subsets](#)

Used when each element can be **taken or skipped**.

Example problems:

- Subsets (LC 78)
- Subsets II (LC 90)

Transition:

```

backtrack(i):
    if i == n: add current subset
    choose to include nums[i]
    choose to exclude nums[i]

```

4.2 [Permutations](#)

Used when order matters.

Example problems:

- Permutations (LC 46)

- Permutations II (LC 47)

Key idea: track used elements.

4.3 [Combinations](#)

Used when selecting k items from n.

Example problems:

- Combinations (LC 77)
 - Combination Sum (LC 39)
 - Combination Sum II (LC 40)
-

4.4 [Constraint Satisfaction](#)

Used when choices must satisfy rules.

Example problems:

- N-Queens
 - Sudoku Solver
 - Word Search
-

5. Example Problem: Generate All Subsets

```
def subsets(nums):
    res = []
    subset = []

    def backtrack(i):
        if i == len(nums):
            res.append(subset.copy())
            return

        # include
        subset.append(nums[i])
        backtrack(i+1)
        subset.pop()

        # exclude
        backtrack(i+1)

    backtrack(0)
    return res
```

6. Common Pitfalls

- Forgetting to **undo** the choice (missing pop())
 - Not handling **duplicates** (sort + skip duplicates)
 - Using global state incorrectly
 - Confusing **permutations** vs **combinations** vs **subsets**
 - Not pruning early when constraints allow it
-

7. Optimization Techniques

- **Pruning:** stop exploring invalid paths early
 - **Sorting:** helps detect duplicates
 - **Boolean used[] arrays:** track visited elements
 - **Early stopping:** break loops when impossible to complete
-

8. Practice Roadmap

Beginner

- Subsets
- Permutations
- Combinations
- Letter Case Permutation

Intermediate

- Combination Sum I/II
- Palindrome Partitioning
- Word Search
- Generate Parentheses

Advanced

- N-Queens
- Sudoku Solver
- Restore IP Addresses

- Word Break II
-

9. Key Takeaways

- Backtracking is **DFS with undoing choices**.
 - Most problems follow a small number of patterns.
 - Pruning and constraints drastically improve performance.
 - Practice recognizing the structure of the search tree.
-

End of document.